DYNAMIC PROGRAMMING

1-DP-Playing with Numbers

```
#include <stdio.h>
int main()
{
    int n;
    scanf("%d", &n);
    long long dp[n + 1];
    dp[0] = 1;
    for (int i = 1; i <= n; i++)
    {
        dp[i] = dp[i - 1];
        if (i >= 3)
            dp[i] += dp[i - 3];
    }
    printf("%lld", dp[n]);
    return 0;
}
```

2-DP-Playing with chessboard

```
#include <stdio.h>
int main()
{
    int n, a[100][100];
```

```
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            scanf("%d", &a[i][j]);
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
        {
            if (i > 0 && j == 0)
                a[i][j] += a[i - 1][j];
            else if (i == 0 && j > 0)
                a[i][j] += a[i][j - 1];
            else if (i > 0 && j > 0)
            {
                if (a[i - 1][j] > a[i][j - 1])
                    a[i][j] += a[i - 1][j];
                else
                    a[i][j] += a[i][j - 1];
            }
        }
    }
    printf("%d", a[n - 1][n - 1]);
    return 0;
}
```

3-DP-Longest Common Subsequence

```
#include <stdio.h>
```

```
#include <string.h>
int main()
{
    char a[1005], b[1005];
    scanf("%s %s", a, b);
    int m = strlen(a), n = strlen(b);
    int dp[1005][1005] = {0};
    for (int i = 1; i <= m; i++)
        for (int j = 1; j <= n; j++)
        {
            if (a[i - 1] == b[j - 1])
                dp[i][j] = dp[i - 1][j - 1] + 1;
            else if (dp[i - 1][j] > dp[i][j - 1])
                dp[i][j] = dp[i - 1][j];
            else
                dp[i][j] = dp[i][j - 1];
        }
    printf("%d", dp[m][n]);
    return 0;
}
```

4-DP-Longest non-decreasing Subsequence

```
#include <stdio.h>
int main()
{
    int n, a[100], dp[100], max = 1;
```

```
    scanf("%d", &n);
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (int i = 0; i < n; i++)
    {
        dp[i] = 1;
        for (int j = 0; j < i; j++)
        {
            if (a[i] >= a[j] && dp[j] + 1 > dp[i])
                dp[i] = dp[j] + 1;
        }
        if (dp[i] > max)
            max = dp[i];
    }
    printf("%d", max);
    return 0;
}
```